# Assignment 1 · Messaging Application

Jendrik Engels · Student no. 202401827

Aarhus University

October 23, 2026

## 1 Introduction

This report documents a small multi-tier messaging application built for the Software Security course. The application lets users exchange private messages, export their own chat history, and (for administrators) restore a chat backup. The protected resource is the set of private messages, each owned by the users who sent or received it. The goal of the project is to build a secure application, challenge its trust assumptions across component boundaries, and then improve it.

The system is composed of three services that run as separate containers and communicate only over HTTP. The Application API authenticates users and coordinates requests, the Message Vault owns the message records, and the Processing Worker runs export and import jobs using a small external helper package. This separation means that authority has to be carried correctly from one service to the next, which is exactly where composition problems tend to appear.

To make the security work concrete, the project ships in two versions. The `code_vulnerable` build contains four deliberately planted vulnerabilities, while the `code_fixed` build remediates each of them. The same tests and the same continuous integration pipeline run against both versions, so the difference between an insecure and a secure build can be observed directly.

## 2 System Design and Trust Boundaries

The application domain is messaging, which was chosen because it has a small number of clearly owned resources and a natural split between an ordinary user role and a privileged administrator role. The functionality is divided across three services so that each one has a single, well defined responsibility and only the permissions it needs. The Application API acts as the interface users interact with, the Message Vault acts as the resource owner, and the Processing Worker performs jobs with the help of an external package. Because the three services run as separate containers and talk to each other over HTTP, every responsibility and every permission is drawn along a visible boundary rather than hidden inside a single process.

Each HTTP call between two services crosses a trust boundary. The user is the outermost untrusted actor and reaches only the Application API. The Message Vault and the Processing Worker are internal services that should never be reached directly by a user. The external `chat_formatter` package is untrusted third party code that is pulled into the Processing Worker, which makes the Worker the least trusted of the three services. A component must not assume that a request arriving at it has already been authenticated or authorized by whoever called it. The composition vulnerability discussed later is a case where that assumption was made and then violated.

Identity is established at the Application API. A user logs in with a username and a password, which are checked against a credential store that keeps only salted password hashes, and a successful login returns a random session token. Authority is carried by that token, which records the user's identity and role and is valid only for a limited period. Role based checks, such as restricting the import operation to administrators, are enforced at the API before any work is started. In the `fixed` build a shared internal service token additionally authenticates the calls between services, so the Worker and the Vault can tell that a request really came from another internal component. Ownership of the protected resource is with the Message Vault, which stores the messages and is responsible for returning only the records that a request is entitled to see.

Table 1 summarises the components and their responsibilities, and Figure 1 shows how a request flows between them.

| Component | Responsibility |
|---|---|
| Application API | Authenticates users, issues and validates session tokens, enforces role-based authorization, and coordinates requests. The only user-facing component. |
| Processing Worker | Runs the export and import jobs using the external `chat_formatter` package, and operates only on the data for a specific job. |
| Message Vault | Owns the message records and enforces access control on every read and write. |
| chat_formatter | External helper package used by the Worker to format exports. |

Table 1: Components and their responsibilities.

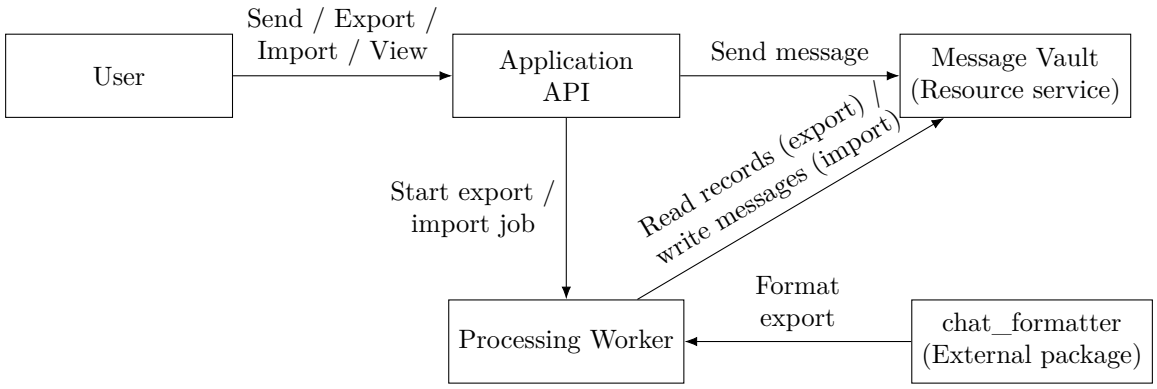



Figure 1: Application workflow and component interactions.

## 3 Authorization Across Components

Authority enters the system at the Application API. When a user logs in, the API checks the supplied username and password against the credential store and, on success, issues a random session token. The token is held on the server and is associated with the user's identity, the user's role, and an expiry time. Every later request carries this token, and the API validates it in one place before any protected action is performed. A request with no token or an unknown token is refused, and a request whose token has passed its expiry time is also refused. Because the same validation runs for every protected endpoint, there is no path that reaches a protected action without passing the check.

The token binds authority to four things. It binds authority to a service, because the session token is only meaningful at the Application API, which is the single component that understands it. It binds authority to an action, because the API checks the user's role before starting a restricted operation, and the import operation is allowed only for an administrator. It binds authority to a resource, because the export operation is scoped to the identity recorded in the token, so a user can only ever export their own inbox and never another user's. Finally it binds authority to a validity period, because the token is rejected once its expiry time has passed, which limits how long a leaked or forgotten token remains useful.

Inside the system the services must also be able to tell that a request really came from another internal component rather than from an outside caller. In the fixed build this is done with a shared internal service token that the Application API includes on its calls to the Processing Worker, and that the Worker in turn includes on its calls to the Message Vault. The Worker and the Vault reject any request that does not present the expected internal token. This is the check that prevents a permitted service from being misused. Even though the Worker is allowed to read records from the Vault as part of an export, an outside caller cannot contact the Worker or the Vault directly, because such a call would not carry a valid session token at the API and would not carry a valid internal token at the Worker or the Vault.

A full export shows how these checks combine, following the flow in Figure 1. The user sends an export request to the Application API with a session token. The API validates the token, confirms that the caller is allowed to run an export, and starts a job on the Processing Worker for the caller's own username, including the internal token. The Worker accepts the request only because the internal token is valid, and asks the Message Vault for the messages belonging to that username, again including the internal token. The Vault accepts the read only because the internal token is valid, and returns only the records for that recipient. The Worker formats the result with the external package and returns it through the API to the user. At no point does a service act on the word of its caller alone.

# 4 Planted Vulnerabilities

The `code_vulnerable` build contains four planted weaknesses, listed in Table 2. Each is remediated in the `code_fixed` build, and the security tests in `tests/test_security.py` demonstrate the difference between the two builds.

| # | Vulnerability |
|---|---|
| 1 | CWE-22 Path Traversal (Application API) |
| 2 | CWE-89 SQL Injection (Message Vault) |
| 3 | CWE-502 Deserialization of Untrusted Data (Processing Worker) |
| 4 | Missing cross-tier authorization / implicit trust between services |

Table 2: Planted vulnerabilities.

## 4.1 CWE-22: Path Traversal

The Application API serves public documents such as the terms of service and the privacy policy through a download endpoint. In the vulnerable build the requested file name is joined to the documents directory without any checking, so a request for a name that contains `../` escapes that directory. A request for `../main.py` returns the source code of the Application API itself, and an absolute path can reach other files inside the container. The impact is the disclosure of files

that the user was never meant to read. The fix resolves the requested path to an absolute path and confirms that it still lies inside the documents directory before serving it, so any attempt to escape the directory is refused.

```
target_dir = "./documents"
# filename is taken from the request and used without any check
file_path = os.path.join(target_dir, filename)
# "../main.py" escapes ./documents, an absolute path ignores
    target_dir
return FileResponse(file_path)
```

### 4.2 CWE-89: SQL Injection

The Message Vault looks up messages by building a SQL query with the requested recipient inserted directly into the query string. A recipient value such as `' OR '1'='1` changes the meaning of the query so that it returns every message instead of the messages for one recipient, which discloses the private messages of all users. This query is reachable only by calling the Vault directly, which ties this weakness to the composition flaw described below. The fix uses a parameterised query, so the recipient value is always treated as data and can no longer alter the query, and the internal service token additionally prevents the Vault from being called by an outside party at all.

```
# recipient is inserted straight into the SQL string, so a value like
    ' OR '1'='1 rewrites the query and returns every row
query = f"SELECT id, sender, recipient, content FROM messages WHERE
   recipient = '{recipient}'"
cursor.execute(query)
```

### 4.3 CWE-502: Deserialization of Untrusted Data

The import feature restores a backup of messages. In the vulnerable build the Processing Worker decodes the backup and deserialises it with `pickle`. Because `pickle` can reconstruct arbitrary objects, a crafted backup runs attacker chosen code inside the Worker as it is deserialised. This is remote code execution in a component that can reach the Message Vault over the internal network, so the impact is severe. The fix parses the backup as JSON instead of unpickling it, and validates that the decoded value is a list of message objects within a size limit, so a backup can only ever be data and can never carry executable code.

```
raw = base64.b64decode(req.backup)
# pickle.loads reconstructs arbitrary objects, so a maliciously
    crafted backup executes code chosen by the attacker during
    deserialization
messages = pickle.loads(raw)
```

### 4.4 Composition: Missing Cross-Tier Authorization

Together the three services trust each other too much. In the vulnerable build the Processing Worker and the Message Vault accept any caller, and their ports are published, so a client can call them directly and skip the Application API entirely. A direct call to the Vault reads any user's inbox, and a direct call to the Worker runs an export for any user, both without a session token and without the administrator role. The composition also makes the SQL injection reachable, since the injectable recipient value can be supplied directly to the Vault. The fix introduces a shared internal service token that the Worker and the Vault require on every call, so a request

that does not come from another internal component is refused. The fixed deployment also binds the internal services to the local interface as an additional layer of isolation.

# 5 Secure Design Principles

## 5.1 Least Privilege

The design decision is that every user and every component is given only the authority it needs and nothing more. An ordinary user may send, export, and view, but only an administrator may import a backup, and the export operation is scoped to the caller's own inbox so one user can never read another user's messages through the Application API. The credential store is also kept in a database that belongs to the Application API and is separate from the message database owned by the Message Vault, so the Vault never holds passwords. This is implemented through a role check on the import endpoint and by deriving the export target from the authenticated identity rather than from user input. The evidence is the authorization test suite, where an ordinary user receives a refusal when calling import and where a message addressed to one user does not appear in another user's export. The separation of the two databases also means that the SQL injection in the Vault can at most reach messages and never reaches credentials.

## 5.2 Complete Mediation

The design decision is that every protected request is checked every time, with no cached decision that could let a later request through unchecked. This is implemented by routing all protected endpoints of the Application API through a single validation function that re-reads the session token, confirms it is known, and confirms it has not expired, on every call. In the fixed build the Worker and the Vault likewise re-check the internal service token on every request they receive. Because there is only one validation point, there is no endpoint that can accidentally skip the check. The evidence is that a request with no token and a request with an unknown token are both refused, and that a token which has passed its validity period is refused on the next call even though it worked before.

## 5.3 Fail-Safe Defaults

The design decision is that access is denied unless it is explicitly granted, so the default outcome of any uncertainty is refusal. This is implemented by having the protected endpoints deny a request whenever a valid session is not present, by having the Worker and the Vault reject any call that does not carry the expected internal token, and by having the document download resolve the requested path and refuse anything that falls outside the documents directory. The evidence is that unauthenticated and direct calls to the internal services are refused, that a missing token yields a refusal rather than access, and that a path traversal attempt is blocked rather than served.

# 6 SSDLC, CI/CD and Gate Decisions

A short lifecycle plan covering the four phases was committed to `docs/Plan.md` before implementation. It defines the components, roles, access control matrix, and STRIDE threats with their controls, and gives each phase a gate decision.

The plan influenced development directly, because the controls it named became the features that were built. The access control matrix fixed which role could perform which operation, which is why import is restricted to an administrator and export is scoped to the caller. The STRIDE entries decided which mechanisms were implemented, so the spoofing control became the session token and its validation, the elevation of privilege control became the role check, and

the intended cross-tier authorization boundary became the internal service token.

The plan influenced the gate decisions in the same way, because each phase gate in the plan became a concrete check in the pipeline.

The pipeline, defined in the `Jenkinsfile`, runs static analysis with Bandit on the build, analyzes dependencies with pip-audit, builds the three Docker images, and runs the functional, authorization, and security tests against the container stack, archiving all reports (copies are kept in `ci-evidence`). A build is blocked when Bandit reports a finding at or above the configured severity and confidence, when the image build fails, or when a functional or authorization test fails. pip-audit still reports advisories against the pinned framework versions in the fixed build, so the dependency gate is treated as a defined policy and the finding is recorded rather than used to block the otherwise secure build.

The two builds give the intended contrast. `code_vulnerable` is red, because Bandit detects the string-formatted SQL query and the use of `pickle` and the static gate stops the run before the later stages. `code_fixed` is green, with clean analysis, a successful build, and all tests passing. The archived run in `ci-evidence/SUCCESS_code-fixed_medium_medium_dont-fail-on-dep-vulns` is the proof of this successful build. It was produced with `TARGET_DIR` set to `code_fixed`, the Bandit severity and confidence thresholds both set to medium, and the dependency gate disabled (`FAIL_ON_DEP_VULNS` unchecked).

The protections themselves come from several layers: the runtime (parameterised queries, JSON instead of `pickle`, salted password hashing), the libraries, the application logic (session validation, role checks, the internal service token, and path confinement), and the deployment (per-container isolation and binding the internal services to the local interface).

## 7 Production Release Considerations

Regarding a production release, several further changes would need to be made. The internal services should not be published at all and should only be reachable on an internal network, so that the service token becomes a second layer of defence rather than the only one. Traffic to the Application API should be protected with TLS, and the calls between services should also be encrypted. The internal service token and the account passwords should come from a secret store rather than from defaults in the compose file or the source, and each environment should use its own secrets.

Beyond the network and secrets, the in-memory session store should be replaced with a durable store that persists across restarts and supports revoking a token before it expires. The two controls that were deferred in the threat model should be added, namely audit logging of privileged actions for accountability and rate limiting at the Application API for availability. Finally, because the dependency analysis shows that advisories accumulate against pinned versions over time, a continuous dependency-update and vulnerability-management process would be needed to keep the dependency gate meaningful. Together these changes would move the system to a deployment that could be operated safely.